

The Gains Are in the Restraint

Search, context, and faithful retrieval-augmented reasoning — a three-paper reader's guide

 Agentic Engineering Papers · catch-up digest, 5 June 2026

June 5, 2026

Chapter 0: The Gains Are in the Restraint

Train a search agent on the obvious reward and it turns into a hoarder. It searches when it already knew the answer, keeps searching after the evidence is in, drags every snippet it ever fetched into context, and then trusts a planted lie on the page over what it learned in pretraining. For a decade the reflex in this field has been to push the same two levers harder: retrieve more, scale the model up. The three papers in this bundle, taken together, make a quieter and more useful argument. The next round of gains in retrieval-augmented systems comes from **restraint** and **grounding** — not from retrieving more. And the right amount of restraint is **regime-dependent**, which means you have to measure before you reach for a technique.

Read in order, the papers trace the full lifecycle of a retrieved fact. SAAS asks *whether to search at all*. Masking Stale Observations asks *what retrieved context to keep*. OCC-RAG asks *how faithfully to use what you kept*. Three decisions, one conviction underneath all of them: the leverage moved off the headline levers and onto discipline.

The conviction they share is that “more” is the wrong default, and that the agent’s own competence is the thing to instrument. **SAAS** shows that a large slice of retrieval spend buys nothing — the agent searches past its own knowledge boundary, and you can teach it where that boundary is, cutting search calls by roughly forty percent with no loss of accuracy. The fix isn’t a flat cap on searching; that just teaches the agent to dodge searches it needed. It’s a penalty that re-estimates the boundary as the policy improves. **Masking Stale Observations** takes the same suspicion to context management and finds something sharper: pruning stale evidence helps in exactly one neighborhood — a strong retriever feeding a mid-capacity model that can’t yet filter noise on its own — and *actively hurts* a strong model with high recall, costing a saturated 120B model nearly five points. The win lives on an inverted-U, not a straight line. **OCC-RAG** closes the loop by betting against scale outright: small models, mid-trained on a synthetic curriculum to reason over context, cite literal quotes, and abstain when the page doesn’t say enough, match or beat general models two to six times their size on faithfulness — and side with the document even when the document is wrong.

Watch for the connective tissue. All three refuse the convenient universal. SAAS won’t hard-code the search threshold because the boundary moves as the agent learns. Masking won’t call pruning a free win because the regime decides the sign of the effect. OCC-RAG won’t accept that faithfulness only emerges at 70B, because it’s a trainable behavior, not a property of size. Each one replaces a cargo-culted heuristic — *search when in doubt, prune the clutter, reach for the biggest model* — with a measurement you take in your own setup first. That’s the throughline worth carrying: instrument the over-search, locate yourself on the inverted-U, test for literal citation and honest abstention as first-class behaviors rather than hoping they emerge from a prompt.

So read these as a sequence, not a grab bag. Chapter 1 teaches an agent to know when *not* to search, and gives you the cheap offline trick — run a question with search off and on, compare — to find the queries your model never needed. Chapter 2 hands you a regime map for when context pruning sharpens reasoning and when it quietly deletes the evidence the agent still wanted, and tells you which two things to measure before you trust it. Chapter 3 puts a small, disciplined reader in the seat you usually fill with a generalist, and argues that the most reusable artifact in the paper isn’t either model — it’s the data recipe you can run against your own corpus. Different decisions, one lesson: the gains are in the restraint, and the restraint has to be measured, not assumed.

Chapter 1: Knowing When Not to Search

Paper: [SAAS: Self-Aware Reinforcement Learning for Over-Search Mitigation in Agentic Search](#) · arXiv 2605.29796 · **Code:** github.com/XMUDeepLIT/SAAS

Train a search agent with the obvious reward — did it get the answer right? — and watch what happens to its habits. Within fifty steps, the fraction of questions it answers without searching at all drops to nearly zero. Searching becomes the default reflex. Meanwhile the share of searches fired *after* the agent already had enough evidence to answer climbs toward fifty percent. The agent isn't getting smarter about retrieval. It's getting compulsive.

That's the problem SAAS is built to fix, and the headline is worth saying plainly: a large chunk of what an agentic-search system spends on retrieval is buying nothing. The model searches when its own parametric knowledge already had the answer, and it keeps searching after the evidence is sufficient. Teach the agent where its own knowledge ends and you recover that wasted cost — latency, compute, and the noise extra passages inject into the answer — without losing accuracy.

Why outcome rewards breed over-search

Agentic search is the loop where an LLM interleaves reasoning with retrieval: think, decide whether to search, pull evidence, fold it back into the reasoning, repeat until it answers. It's how these systems crack multi-hop questions that no single lookup can resolve.

The trouble is the training signal. Because external search usually *helps* on hard questions, an outcome-only reward keeps nudging the policy toward searching more — but it says nothing about whether any individual search was *necessary*. So the agent never learns its **knowledge boundary**: the line between questions it can answer from memory and questions that genuinely need evidence. The paper names two failure modes on either side of it — **unnecessary search** (querying when parametric knowledge already suffices) and **redundant search** (continuing to query after enough evidence is in hand).

The naive fix — penalize every search past some threshold — fails for an instructive reason. A fixed penalty doesn't just trim redundant searches; it collapses training around step 250, dragging accuracy *below* the no-penalty baseline, because the agent games the penalty by skipping searches it actually needed. Worse, the boundary itself moves: as the policy improves, the share of questions answerable without search rose from about 13% to 24% over training. A static threshold is aiming at a target that won't hold still.

Three moves that make self-awareness work

SAAS — Self-Aware reinforcement learning for Agentic Search — replaces the fixed threshold with a boundary the system re-estimates as the policy evolves. Three components carry the load.

First, **search boundary modeling**. For each question, the agent runs two groups of rollouts under the current policy: one *search-disabled* (reasoning from memory only) and one *search-enabled*. Counting how many correct answers each group produces tells you which side of the boundary the question sits on right now. Reliably solvable without search? Tag it `NoSearch`. Only solvable with retrieval? `NeedSearch`. Ambiguous? `Undetermined`, and leave it alone. This is measured *on-policy* — the ablation shows freezing the boundary at the pre-training base model costs nearly three accuracy points, because the boundary genuinely shifts as the agent learns.

Second, a **boundary-aware reward** that turns those tags into trajectory-level penalties. `NoSearch` questions get zero tolerance — any search is penalized. `NeedSearch` questions don't penalize searching at all; instead they estimate a minimum sufficient search count (the fewest searches among correct trajectories) and penalize only the queries beyond it. The penalty fires only on fully correct trajectories, so the agent is never punished for searching its way to a right answer — only for the searches that didn't change the outcome.

Third, a **stage-wise curriculum** that sequences the training. Stage one uses the plain outcome reward so the agent first learns to reason and use the search tool competently. Only when validation plateaus does stage two switch on the boundary-aware penalty. Flip it on too early and you get reward hacking — the agent learns to avoid searching before it ever learned to search well. The ablation confirms it: drop the staging and average accuracy falls from 45.8% to 40.9%.

Reading the result

The payoff is a cleaner accuracy-efficiency trade than the strong baselines reach. On Qwen2.5-3B, SAAS lifts accuracy over the best prior method while cutting search calls by roughly forty percent. The over-search metrics separate most sharply: search-heavy RL baselines like StepSearch and HiPRAG drive their question-level over-search ratio to essentially 100% — searching on nearly every answerable-from-memory question — while SAAS holds it to about 46% and pushes step-level redundancy into single digits. Same or better answers, far fewer searches.

The broader lesson reaches past this one framework. We tend to treat retrieval as free insurance: when in doubt, search. SAAS is evidence that the doubt itself is the expensive part, and that an agent can be taught to price it. The win didn't come from a better retriever or a sharper prompt — it came from giving the policy a signal about its own competence and letting that signal evolve.

What could you try? Before changing anything, instrument your search agent for over-search: log the searches that didn't move the final answer, and the queries issued after the evidence was already sufficient. That measurement alone usually reveals more waste than people expect. If it does, add a

boundary-aware penalty rather than a flat cap — punish the searches that didn't earn their keep, not searching in general, and re-estimate the boundary as your agent improves instead of hard-coding it once. And sequence the training: let the agent get good at retrieval first, then clamp down, or you'll teach it to dodge searches it actually needs. Even outside RL, the contrastive trick — run a question with search off and with search on, then compare success rates — is a cheap offline way to find which queries your model never needed.

A search agent's other recurring tax is what it drags along: every observation it ever retrieved, piling up in context turn after turn. The next chapter looks at masking those stale observations — pruning old retrieved evidence out of the agent's window to keep it focused — and, more usefully, at a regime map of when that context-management trick sharpens reasoning and when it quietly throws away the very evidence the agent still needed.

Chapter 2: Forgetting on Purpose

Paper: [Masking Stale Observations Helps Search Agents – Until It Doesn’t: A Regime Map and Its Mechanism](#) · arXiv 2606.00408 · **Code:** github.com/i-DeepSearch/observation-masking

A long-horizon search agent is a hoarder. Every turn, it appends another batch of retrieved snippets, page dumps, and tool errors to its running context. By the time it answers, the trajectory can run to tens of thousands of tokens, and more than 85% of those tokens are raw environment observations the agent fetched along the way. The obvious fix is to throw old stuff out: **observation masking** replaces stale tool outputs with a placeholder while keeping the reasoning and tool-call structure intact. It costs nothing extra and ships in real deployed agents. So you’d expect a clean story — prune the clutter, win some accuracy. This paper’s central finding is that the story isn’t clean at all.

The win is real, but only in one neighborhood

The authors ran a wide sweep: agent backbones from 4B to 284B parameters, three retrievers spanning sparse BM25 to a dense retriever tuned for agentic search, across offline and live-web benchmarks. They held everything else fixed — same scaffold, same masking rule, same prompts — so the gains they measured reflect the intervention, not lucky configuration.

Plot the accuracy gain from masking against the model’s accuracy *without* any context management, and you get an **asymmetric inverted-U**. Three regimes fall out of it:

- **The retriever bottleneck (a plateau).** Under weak retrievers like BM25, recall never clears 0.55, and masking buys a modest, flat +6 to +7 points no matter how big the backbone is. There simply isn’t enough answer-supporting evidence in the context for pruning to amplify. The retriever caps accuracy, not the model.
- **The sweet spot (the peak).** The biggest win — +11.7 points — landed where a strong retriever (recall 0.88) met a *mid-capacity* model whose no-management accuracy was still middling (around 63%). The retriever surfaced the evidence; the model couldn’t yet separate signal from noise on its own. Masking did that separation for it.
- **The collapse.** Once a model’s baseline accuracy passed about 70%, the gain cratered — bottoming at **−1.1 points** for the strongest model in the table, which had the highest recall and the highest baseline accuracy and was actively *harmed* by masking. On live-web GAIA, a saturated 120B model lost a full −4.8 points.

The headline isn’t “masking helps” or “masking hurts.” It’s that the outcome is governed by the *interaction* between retriever recall and the model’s own filtering capacity — not either one alone. And it’s a mismatch effect, not a size effect: two models with identical architecture and parameter count sat in completely different regimes (+11.7 vs. +3.7) purely because of how their training shaped their attention.

A token-for-turn trade

Mechanistically, the paper reframes masking as a trade: you spend evidence you're probably not using to buy context budget for more turns. Two lines of evidence back this up.

First, attention. When the authors measured where the model actually looks, **reasoning carried the attention mass, not observations** — 53.7% of the per-step budget went to the model's own reasoning chains versus 25.6% to tool observations. Worse for the hoarding instinct, observation attention is front-loaded: it collapses to near-zero just one turn back and stays pinned there. The model treats observations as transient inputs to consume once. Masking targets exactly the mid-trajectory content the model has already stopped reading.

Second, re-opening behavior. Agents rarely page back to the middle of their history; they re-open either the page they just fetched or the very first one. Masking sharpens that bimodal habit. It nudges weaker models to anchor back to their initial plan the way stronger models already do — sometimes by actively re-searching for something it dropped.

That's why masking helps *and* why it backfires. When the freed-up turns convert failures into successes, you win cheaply. But when masking evicts evidence the model *would* have used, the agent pays dearly to re-search and re-open pages. In the sweet-spot model, fixes outnumbered breaks roughly 3:1. In the saturated model, they ran about 1:1 — masking helped and harmed in equal measure, and the gains simply canceled.

What this means, and what to try

The practical lesson cuts against a comfortable default. Context pruning is treated in a lot of agent stacks as a free, universal win — strip the clutter, keep the budget lean. This paper says that's only true in one regime, and you can predict which regime you're in *before* you trust the heuristic. If you pair a strong model with a strong retriever, masking may actively hurt you. The room for improvement only exists when the retriever is feeding good evidence the model can't yet filter.

So before you adopt observation masking — or any aggressive context-pruning heuristic — locate yourself on the inverted-U. Measure two things separately: your retriever's recall against the evidence you actually need, and your model's own ability to ignore noise (a proxy: how much of the answer's supporting evidence already surfaces early in the trajectory). Weak retriever? Pruning won't save you; fix retrieval first. Strong retriever, mid-capacity model? That's the band where masking earns its keep. Strong model with high recall already? Leave the context alone — you'll mostly be deleting things the model was going to use. The broader takeaway the authors push: as backbones get more capable, the engineering leverage moves away from clever pruning and toward higher-fidelity retrieval, because better retrieval raises the ceiling on actionable signal in a way that throwing tokens away never can.

That points naturally to the opposite design philosophy in the next chapter. Rather than trimming context after the fact from a general-purpose backbone, OCC-RAG builds small, task-specialized models from the ground up for faithful, citation-grounded, abstention-aware multi-hop question answering — models tuned to know what they know, cite where they got it, and decline when the evidence isn't there, matching or beating models two to six times their size.

Chapter 3: Small Models That Only Trust the Page in Front of Them

Paper: [OCC-RAG: Optimal Cognitive Core for Faithful Question Answering](#) · arXiv 2606.00683 ·

Models: huggingface.co/occ-ai

Here is a small puzzle the OCC team puts at the center of their paper. You feed a model a passage that says, plainly, that Charles de Gaulle was elected the first U.S. president in 2022. Then you ask: who was the first president of the United States? A 70-billion-parameter model answers “Charles de Gaulle” — it trusts the page. An 8B model answers “George Washington” — correct about the world, wrong about the document. A 1B model just hallucinates “Donald Trump.” And a 1.7B model trained specifically for this job answers “Charles de Gaulle,” siding with the larger, faithful model rather than its own memory.

That last model is the point. For a whole class of applications, the thing you want is not a model that knows more — it’s a model that reasons carefully over the text you handed it and refuses to wander off into what it memorized during pretraining.

The bet against scale

The last several years of language-model progress have been a story about **scale**: each generation stuffs more of the world’s knowledge into its weights. That makes sense when you want a model that can answer anything. But a lot of real systems — retrieval pipelines, document QA, anything where an answer has to be defensible against a source — don’t need a walking encyclopedia. They need disciplined reading.

OCC, short for **Optimal Cognitive Core**, is a family of small language models built on the opposite bet: prioritize task-specific reasoning over parametric knowledge. The variant here, OCC-RAG, is tuned for faithful question answering grounded in provided context — multi-hop reasoning over supplied passages while deliberately ignoring what it happens to remember. (The “RAG” in the name signals the deployment setting, even though the model embeds no retriever of its own.)

They built and released two checkpoints, **OCC-RAG-0.6B** and **OCC-RAG-1.7B**, mid-trained from Qwen3 base models. The headline result: these compact, specialized models match or exceed general-purpose models 2 to 6 times their size across multi-hop reasoning (HotpotQA, MuSiQue, TAT-QA), context faithfulness (ConFiQA), and refusal on unanswerable questions (MuSiQue-Un). The 0.6B model beats Gemma-3-4B and SmoLLM-3-3B on every dimension, and on faithfulness both OCC models post the best scores at any scale tested up to 32B — the lowest rate of defaulting to memorized answers when the context contradicts them.

The recipe is the data, not the architecture

The interesting engineering here is not a clever model design — it’s a data pipeline. The team built a synthetic corpus of over 3 million multi-context, multi-hop QA examples (about 3.25M pairs, roughly 8B tokens) engineered to teach three specific behaviors at once.

The first is **multi-hop reasoning**. To generate questions that genuinely require stitching facts across passages — not just span extraction — they extract a knowledge graph from the source text and sample explicit reasoning paths through it. Conditioning generation on a fixed path guarantees the “bridge” entities a real multi-hop question needs, and pins the gold answer to the path itself rather than trusting the generator to be right.

The second is **strict context faithfulness**. Every answer is recoverable from the supplied passages alone, and each example is salted with semantically similar distractor passages so the model can’t lean on surface cues — distractors, tellingly, eat the largest share of the token budget.

The third is **calibrated abstention**. About 43,000 examples are deliberately unanswerable, with “Not enough information” as the correct response. They build these hard refusal cases by checking whether a strong extractive model can still recover the answer from a reduced context; if it can’t, a critical fact is missing and the model should decline.

Crucially, every example carries an explicit reasoning trace with a fixed shape: Query Analysis, a Source Analysis that attributes facts to numbered sources, a Reasoning section that combines them, and — the team’s own addition — a **Status** verdict that commits to ANSWERABLE or UNANSWERABLE before the final answer appears. Making abstention a discrete label the model must commit to, rather than something it backs into through phrasing, turns the answerable-vs-not boundary into a real supervised target. And because the output cites sources with literal quotes from the context, you get chain-of-thought-level transparency at a fraction of the cost of full thinking-mode inference.

What this means, and what you could try

So what does a result like this change? It pushes back on the reflex to reach for the biggest general model whenever a task feels hard. For grounded QA, “hard” often means “requires careful reading and honest abstention” — and those are trainable behaviors, not properties that only emerge at 70B. The team’s own framing is the sharp version: faithfulness does not require scale; it can be learned through the right curriculum and supervision format. A 0.6B model that quotes its sources and knows when to say “not enough information” can be both cheaper and more trustworthy than a generalist ten times its size that confidently overrides the document.

If you ship a RAG or document-QA product, the concrete move is to stop treating the reader as a commodity slot for whatever big model you happen to have. Consider a small specialized **cognitive core** in that seat instead — one trained to reason over context, cite literally, and abstain. Make literal-

quote citations and calibrated abstention first-class behaviors you actually test for, not nice-to-haves you hope emerge from prompting. And if you have a domain the released checkpoints don't cover, the most reusable artifact in this paper isn't either model — it's the data recipe. Synthesizing multi-context, multi-hop QA with built-in distractors, faithfulness targets, and explicit unanswerable cases is a procedure you can run against your own corpus to fine-tune a grounded reader that behaves the way you need.

That closes the loop on this bundle's through-line. The earlier chapters pushed on how systems retrieve and how they reason over what they pull in; OCC-RAG lands on the quieter discipline underneath both — trusting the page in front of you, citing it honestly, and admitting when it doesn't say enough. Retrieval-augmented reasoning, in the end, is less about how much a model knows than about how faithfully it reads. And faithful reading, this work suggests, is something you can teach a small model to do well.